I've censored some photos in the PDF for privacy reasons. If you'd like the complete .ipynb file, feel free to email me at parthgupta9999@gmail.com — I'll be happy to share it with you.

cihpfkekm

April 11, 2025

[2]:
```
import tensorflow as tf
from tensorflow.keras import models, layers
import matplotlib.pyplot as plt
```

[3]:
```
IMAGE_SIZE = 256
BATCH_SIZE = 32
CHANNELS = 3
EPOCHS = 50
```

[4]:
```
dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "data",
    shuffle = True,
    image_size = (IMAGE_SIZE, IMAGE_SIZE),
    batch_size = BATCH_SIZE
)
```

```
Found 128 files belonging to 3 classes.
```

[5]:
```
class_names = dataset.class_names
class_names
```

[5]:
```
['Mansi', 'Parth', 'Premlata']
```

[6]:
```
plt.figure(figsize=(10,10))
for image_batch, label_batch in dataset.take(1):
    for i in range(12):
        ax = plt.subplot(3,4,i+1)
        plt.imshow(image_batch[i].numpy().astype('uint8'))
        plt.title(class_names[label_batch[i]])
        plt.axis("off")
```

[14]:
```
def get_dataset_partition_tf(ds, train_split=0.8, val_split=0.1,test_split=0.1,␣
 ↪shuffle=True, shuffle_size = 10000):
    ds_size = len(ds)
    if shuffle:
        ds = ds.shuffle(shuffle_size, seed =12)
    train_size = int(train_split*ds_size-1)
    val_size = int(val_split*ds_size+1)
    train_ds = ds.take(train_size)
    val_ds = ds.skip(train_size).take(val_size)
    test_ds = ds.skip(train_size).skip(val_size)

    return(train_ds, val_ds, test_ds)
```

[15]:
```
train_ds, val_ds, test_ds = get_dataset_partition_tf(dataset)
```

[16]:
```
len(train_ds)
```

[16]: 2

[17]:
```
len(val_ds)
```

[17]: 1

[18]:
```
len(test_ds)
```

[18]: 1

[19]:
```
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size = tf.data.
 ↪AUTOTUNE)
val_ds = val_ds.cache().shuffle(1000).prefetch(buffer_size = tf.data.AUTOTUNE)
test_ds = test_ds.cache().shuffle(1000).prefetch(buffer_size = tf.data.AUTOTUNE)
```

[20]:
```
resize_and_rescale = tf.keras.Sequential([
    layers.experimental.preprocessing.Resizing(IMAGE_SIZE, IMAGE_SIZE),
    layers.experimental.preprocessing.Rescaling(1.0/255)
])
```

[21]:
```
data_augmentation = tf.keras.Sequential([
    layers.experimental.preprocessing.RandomFlip("horizontal_and_vertical"),
    layers.experimental.preprocessing.RandomRotation(0.2)
])
```

[22]:
```
input_shape = (BATCH_SIZE, IMAGE_SIZE, IMAGE_SIZE, CHANNELS)
n_classes = 3

model = models.Sequential([
    resize_and_rescale,
    data_augmentation,
    layers.Conv2D(32, kernel_size = (3,3), activation='relu',␣
 ↪input_shape=input_shape),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64,  kernel_size = (3,3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64,  kernel_size = (3,3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
```

```
    layers.Dense(64, activation='relu'),
    layers.Dense(n_classes, activation='softmax'),
])

model.build(input_shape=input_shape)
```

[23]:
```
model.summary()
```

```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
sequential (Sequential)      (32, 256, 256, 3)         0
_________________________________________________________________
sequential_1 (Sequential)    (32, 256, 256, 3)         0
_________________________________________________________________
conv2d (Conv2D)              (32, 254, 254, 32)        896
_________________________________________________________________
max_pooling2d (MaxPooling2D) (32, 127, 127, 32)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (32, 125, 125, 64)        18496
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (32, 62, 62, 64)          0
_________________________________________________________________
conv2d_2 (Conv2D)            (32, 60, 60, 64)          36928
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (32, 30, 30, 64)          0
_________________________________________________________________
conv2d_3 (Conv2D)            (32, 28, 28, 64)          36928
_________________________________________________________________
max_pooling2d_3 (MaxPooling2 (32, 14, 14, 64)          0
_________________________________________________________________
conv2d_4 (Conv2D)            (32, 12, 12, 64)          36928
_________________________________________________________________
max_pooling2d_4 (MaxPooling2 (32, 6, 6, 64)            0
_________________________________________________________________
conv2d_5 (Conv2D)            (32, 4, 4, 64)            36928
_________________________________________________________________
max_pooling2d_5 (MaxPooling2 (32, 2, 2, 64)            0
_________________________________________________________________
flatten (Flatten)            (32, 256)                 0
_________________________________________________________________
dense (Dense)                (32, 64)                  16448
_________________________________________________________________
dense_1 (Dense)              (32, 3)                   195
=================================================================
Total params: 183,747
```

```
Trainable params: 183,747
Non-trainable params: 0
_________________________________________________________________
```

[24]:
```
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy']
)
```

[25]:
```
history = model.fit(
    train_ds,
    batch_size=BATCH_SIZE,
    validation_data=val_ds,
    verbose=1,
    epochs=50,
)
```

```
Epoch 1/50
2/2 [==============================] - 4s 2s/step - loss: 1.0888 - accuracy:
0.4062 - val_loss: 1.0498 - val_accuracy: 0.4688
Epoch 2/50
2/2 [==============================] - 2s 1s/step - loss: 1.0679 - accuracy:
0.4062 - val_loss: 1.0547 - val_accuracy: 0.4688
Epoch 3/50
2/2 [==============================] - 2s 1s/step - loss: 1.0570 - accuracy:
0.4062 - val_loss: 1.0237 - val_accuracy: 0.4688
Epoch 4/50
2/2 [==============================] - 2s 1s/step - loss: 1.0410 - accuracy:
0.4062 - val_loss: 0.9820 - val_accuracy: 0.4688
Epoch 5/50
2/2 [==============================] - 2s 1s/step - loss: 0.9946 - accuracy:
0.5156 - val_loss: 0.9015 - val_accuracy: 0.6875
Epoch 6/50
2/2 [==============================] - 2s 1s/step - loss: 0.9182 - accuracy:
0.6094 - val_loss: 0.7363 - val_accuracy: 0.8438
Epoch 7/50
2/2 [==============================] - 2s 1s/step - loss: 0.7582 - accuracy:
0.7969 - val_loss: 0.6697 - val_accuracy: 0.5625
Epoch 8/50
2/2 [==============================] - 2s 1s/step - loss: 0.7325 - accuracy:
0.7031 - val_loss: 0.6837 - val_accuracy: 0.6562
Epoch 9/50
2/2 [==============================] - 2s 1s/step - loss: 0.5701 - accuracy:
0.7656 - val_loss: 0.3363 - val_accuracy: 0.8750
Epoch 10/50
2/2 [==============================] - 2s 1s/step - loss: 0.4217 - accuracy:
0.8906 - val_loss: 0.4402 - val_accuracy: 0.8438
```

```
Epoch 11/50
2/2 [==============================] - 3s 1s/step - loss: 0.3530 - accuracy:
0.8906 - val_loss: 0.1418 - val_accuracy: 0.9688
Epoch 12/50
2/2 [==============================] - 2s 1s/step - loss: 0.2409 - accuracy:
0.9062 - val_loss: 0.0894 - val_accuracy: 0.9688
Epoch 13/50
2/2 [==============================] - 2s 1s/step - loss: 0.1117 - accuracy:
1.0000 - val_loss: 0.1255 - val_accuracy: 1.0000
Epoch 14/50
2/2 [==============================] - 2s 1s/step - loss: 0.0849 - accuracy:
1.0000 - val_loss: 0.0218 - val_accuracy: 1.0000
Epoch 15/50
2/2 [==============================] - 2s 1s/step - loss: 0.0438 - accuracy:
0.9844 - val_loss: 0.0209 - val_accuracy: 1.0000
Epoch 16/50
2/2 [==============================] - 2s 1s/step - loss: 0.0221 - accuracy:
1.0000 - val_loss: 0.0187 - val_accuracy: 1.0000
Epoch 17/50
2/2 [==============================] - 2s 1s/step - loss: 0.0149 - accuracy:
1.0000 - val_loss: 0.0089 - val_accuracy: 1.0000
Epoch 18/50
2/2 [==============================] - 2s 1s/step - loss: 0.0060 - accuracy:
1.0000 - val_loss: 0.0100 - val_accuracy: 1.0000
Epoch 19/50
2/2 [==============================] - 2s 1s/step - loss: 0.0092 - accuracy:
1.0000 - val_loss: 0.0104 - val_accuracy: 1.0000
Epoch 20/50
2/2 [==============================] - 2s 1s/step - loss: 0.0077 - accuracy:
1.0000 - val_loss: 0.0025 - val_accuracy: 1.0000
Epoch 21/50
2/2 [==============================] - 2s 1s/step - loss: 0.0131 - accuracy:
0.9844 - val_loss: 0.0030 - val_accuracy: 1.0000
Epoch 22/50
2/2 [==============================] - 2s 1s/step - loss: 8.1631e-04 - accuracy:
1.0000 - val_loss: 0.0097 - val_accuracy: 1.0000
Epoch 23/50
2/2 [==============================] - 2s 1s/step - loss: 0.0057 - accuracy:
1.0000 - val_loss: 0.0073 - val_accuracy: 1.0000
Epoch 24/50
2/2 [==============================] - 2s 1s/step - loss: 0.0044 - accuracy:
1.0000 - val_loss: 0.0022 - val_accuracy: 1.0000
Epoch 25/50
2/2 [==============================] - 2s 1s/step - loss: 0.0035 - accuracy:
1.0000 - val_loss: 3.7256e-04 - val_accuracy: 1.0000
Epoch 26/50
2/2 [==============================] - 2s 1s/step - loss: 2.2553e-04 - accuracy:
1.0000 - val_loss: 8.7756e-05 - val_accuracy: 1.0000
```

```
Epoch 27/50
2/2 [==============================] - 2s 1s/step - loss: 5.3568e-04 - accuracy:
1.0000 - val_loss: 3.9673e-05 - val_accuracy: 1.0000
Epoch 28/50
2/2 [==============================] - 2s 1s/step - loss: 0.0087 - accuracy:
1.0000 - val_loss: 1.4110e-04 - val_accuracy: 1.0000
Epoch 29/50
2/2 [==============================] - 2s 1s/step - loss: 0.0036 - accuracy:
1.0000 - val_loss: 0.0025 - val_accuracy: 1.0000
Epoch 30/50
2/2 [==============================] - 2s 1s/step - loss: 0.0022 - accuracy:
1.0000 - val_loss: 0.0335 - val_accuracy: 1.0000
Epoch 31/50
2/2 [==============================] - 2s 1s/step - loss: 0.0076 - accuracy:
1.0000 - val_loss: 0.0021 - val_accuracy: 1.0000
Epoch 32/50
2/2 [==============================] - 2s 1s/step - loss: 3.3787e-04 - accuracy:
1.0000 - val_loss: 0.0044 - val_accuracy: 1.0000
Epoch 33/50
2/2 [==============================] - 2s 1s/step - loss: 0.0073 - accuracy:
1.0000 - val_loss: 0.0053 - val_accuracy: 1.0000
Epoch 34/50
2/2 [==============================] - 2s 1s/step - loss: 8.3787e-04 - accuracy:
1.0000 - val_loss: 0.0076 - val_accuracy: 1.0000
Epoch 35/50
2/2 [==============================] - 2s 1s/step - loss: 8.2196e-04 - accuracy:
1.0000 - val_loss: 0.0437 - val_accuracy: 1.0000
Epoch 36/50
2/2 [==============================] - 2s 1s/step - loss: 0.0042 - accuracy:
1.0000 - val_loss: 0.0143 - val_accuracy: 1.0000
Epoch 37/50
2/2 [==============================] - 2s 1s/step - loss: 0.0020 - accuracy:
1.0000 - val_loss: 0.0099 - val_accuracy: 1.0000
Epoch 38/50
2/2 [==============================] - 2s 1s/step - loss: 5.2821e-04 - accuracy:
1.0000 - val_loss: 0.0062 - val_accuracy: 1.0000
Epoch 39/50
2/2 [==============================] - 2s 1s/step - loss: 2.2887e-04 - accuracy:
1.0000 - val_loss: 0.0043 - val_accuracy: 1.0000
Epoch 40/50
2/2 [==============================] - 2s 1s/step - loss: 7.4449e-04 - accuracy:
1.0000 - val_loss: 0.0035 - val_accuracy: 1.0000
Epoch 41/50
2/2 [==============================] - 2s 1s/step - loss: 7.1333e-05 - accuracy:
1.0000 - val_loss: 0.0030 - val_accuracy: 1.0000
Epoch 42/50
2/2 [==============================] - 2s 1s/step - loss: 1.8855e-04 - accuracy:
1.0000 - val_loss: 0.0023 - val_accuracy: 1.0000
```

```
Epoch 43/50
2/2 [==============================] - 2s 1s/step - loss: 1.5089e-04 - accuracy:
1.0000 - val_loss: 0.0018 - val_accuracy: 1.0000
Epoch 44/50
2/2 [==============================] - 2s 1s/step - loss: 3.8162e-04 - accuracy:
1.0000 - val_loss: 0.0017 - val_accuracy: 1.0000
Epoch 45/50
2/2 [==============================] - 2s 1s/step - loss: 1.4767e-04 - accuracy:
1.0000 - val_loss: 0.0016 - val_accuracy: 1.0000
Epoch 46/50
2/2 [==============================] - 2s 1s/step - loss: 2.2414e-04 - accuracy:
1.0000 - val_loss: 0.0014 - val_accuracy: 1.0000
Epoch 47/50
2/2 [==============================] - 2s 1s/step - loss: 1.4575e-04 - accuracy:
1.0000 - val_loss: 0.0012 - val_accuracy: 1.0000
Epoch 48/50
2/2 [==============================] - 2s 1s/step - loss: 8.4566e-04 - accuracy:
1.0000 - val_loss: 6.7451e-04 - val_accuracy: 1.0000
Epoch 49/50
2/2 [==============================] - 2s 1s/step - loss: 1.9703e-04 - accuracy:
1.0000 - val_loss: 4.7078e-04 - val_accuracy: 1.0000
Epoch 50/50
2/2 [==============================] - 2s 1s/step - loss: 1.0389e-04 - accuracy:
1.0000 - val_loss: 3.4795e-04 - val_accuracy: 1.0000
```

[26]:
```
scores = model.evaluate(test_ds)
```

```
1/1 [==============================] - 0s 461ms/step - loss: 3.4773e-04 -
accuracy: 1.0000
```

[27]:
```
scores
```

[27]:
```
[0.00034772921935655177, 1.0]
```

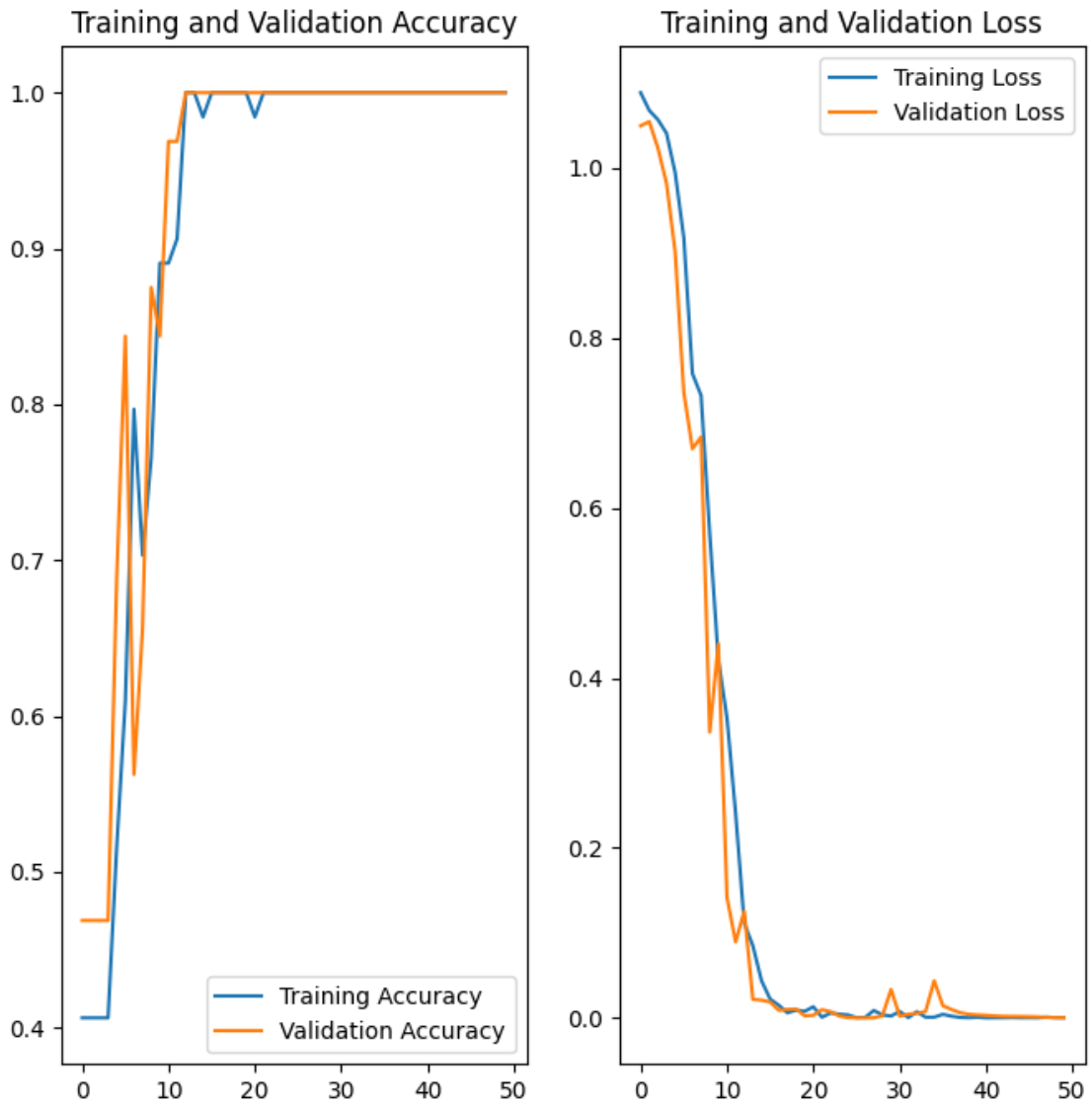
[28]:
```
history.history.keys()
```

[28]:
```
dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

[29]:
```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
print(acc)
loss = history.history['loss']
val_loss = history.history['val_loss']
```

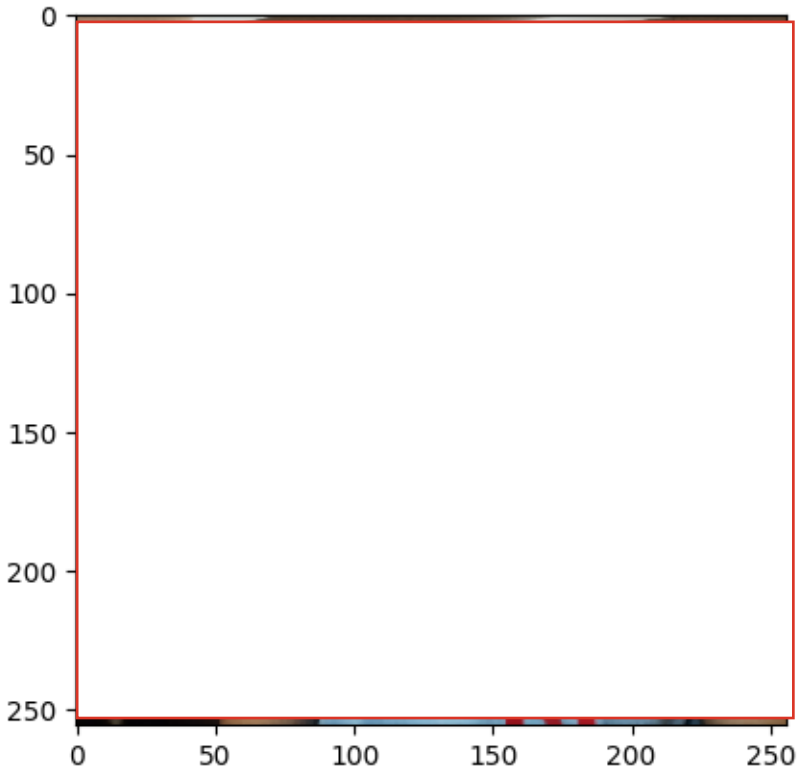
```
[0.40625, 0.40625, 0.40625, 0.40625, 0.515625, 0.609375, 0.796875, 0.703125,
0.765625, 0.890625, 0.890625, 0.90625, 1.0, 1.0, 0.984375, 1.0, 1.0, 1.0, 1.0,
1.0, 0.984375, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

[30]:
```
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(range(EPOCHS), acc, label='Training Accuracy')
plt.plot(range(EPOCHS), val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(range(EPOCHS), loss, label='Training Loss')
plt.plot(range(EPOCHS), val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```



[31]:
```
import numpy as np
for images_batch, labels_batch in test_ds.take(1):

    first_image = images_batch[0].numpy().astype('uint8')
    first_label = labels_batch[0].numpy()

    print("first image to predict")
    plt.imshow(first_image)
    print("actual label:",class_names[first_label])

    batch_prediction = model.predict(images_batch)
    print("predicted label:",class_names[np.argmax(batch_prediction[0])])
```

```
first image to predict
actual label: Parth
predicted label: Parth
```



[32]:
```
def predict(model, img):
    img_array = tf.keras.preprocessing.image.img_to_array(images[i].numpy())
    img_array = tf.expand_dims(img_array, 0) # Create a batch
```

```
    predictions = model.predict(img_array)

    predicted_class = class_names[np.argmax(predictions[0])]
    confidence = round(100 * (np.max(predictions[0])), 2)
    return predicted_class, confidence
```

[ ]:

[ ]:

[ ]:

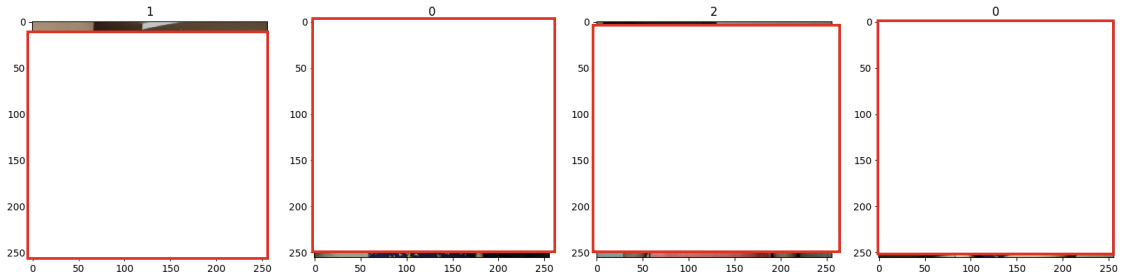
[33]:
```
plt.figure(figsize=(15, 15))
for images, labels in test_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))

        predicted_class, confidence = predict(model, images[i].numpy())
        actual_class = class_names[labels[i]]

        plt.title(f"Actual: {actual_class},\n Predicted: {predicted_class}.\n␣
 ↪Confidence: {confidence}%")

        plt.axis("off")
```

[ ]:
```
model.save("family.h5")
```

[5]:
```
data_iterator=dataset.as_numpy_iterator()
```

[6]:
```
batch = data_iterator.next()
```

[7]:
```
fig, ax = plt.subplots(ncols=4, figsize=(20,20))
for idx, img in enumerate(batch[0][:4]):
    ax[idx].imshow(img.astype(int))
    ax[idx].title.set_text(batch[1][idx])
```



[ ]:

[ ]:

I've censored some photos in the PDF for privacy reasons. If you'd like the complete .ipynb file, feel free to email me at parthgupta9999@gmail.com — I'll be happy to share it with you.